

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

***** ১। Deadlock (ডেডলক) কী?**

DUET: 2007-08, 13-14, BCS 38th বাকশির্বো- ২০০৩, ০৪, ০৭, ০৮, ১১, ১২, ১৩, ১৪'পরী, ১৫, ১৬, ১৭, ১৮, ১৯, ২০, ২৩, ২৪'পরী **NTRCA- 2014, 16, BCPCL-19, BPSC-19**

উত্তরঃ একটি Resource কে যখন একাধিক প্রোগ্রাম বা Process ব্যবহার করতে চায় তখন যে অবস্থার সৃষ্টি হয়, অপারেটিং সিস্টেমের ভাষায় তাকে ডেডলক বলে।

***** ২। Deadlock এর শর্ত লেখ।**

DPDC-17, BREB-19, BTCL-24, বাকশির্বো- ২০০৬, ১২, ১৪'পরী, ১৫, ১৬'পরী, ২০'পরী, ২১, ২৪

উত্তরঃ Deadlock এর শর্তগুলো হলো :

- | | |
|---------------------|-------------------|
| i) Mutual Exclusion | ii) Hold & Wait |
| iii) No-preemption | iv) Circular Wait |

*** ৩। Roll back বলতে কী বুঝ?

বাকশির্বো- ২০০৫, ০৯, ১০, ১১, ১৭, ১৮, ২৩

উত্তরঃ Roll back বলতে কোনো Execution এর সময় কোনো কারণে কোনো Error বা সমস্যা হলে তা পূর্বের অবস্থায় ফিরিয়ে নেওয়ার প্রক্রিয়াকে বুঝায়। এর ফলে ডাটার নিরাপত্তা এবং সিস্টেমের স্থিতিশীলতা নিশ্চিত হয়।

যেমন : Windows System Restore, Image Backup ইত্যাদি।

** ৪। Deadlock Detection এর বিবেচ্য বিষয় কয়টি ও কী কী?

বাকশির্বো- ২০১২, ১৮, ২০

অথবা, Deadlock Detection এর কাজ কী?

উত্তরঃ কোনো System এ যদি Deadlock Prevention প্রয়োগ না করা হয়, তবে Deadlock তৈরি হতে পারে। তাই দুটি কাজ করা জরুরি। যথা :

- i) System এ Deadlock তৈরি হলো কিনা তা দেখা এবং
- ii) Deadlock ঘটে থাকলে তা Recover করা।

*** ৫। কী কী কারণে Deadlock হয়?

বাকশির্বো- ২০০৯, ১৭, ১৮

উত্তরঃ চারটি কারণে একটি সিস্টেমে Deadlock হতে পারে। যথা :

- i) Mutual Exclusion
- ii) Hold and Wait
- iii) No-Preemption এবং
- iv) Circular Wait

SOS

সংক্ষিপ্ত প্রশ্নোত্তর :

*** ১। **Deadlock**-এর শর্তাবলি বর্ণনা কর।

DUET: 2007-08, NTRCA-2014, বাকাশিৰো- ২০০৭, ০৮, ১০, ১২, ১২'পরি, ১৫, ১৭'পরি, ১৯'পরি, ২৪'পরি

উত্তরঃ একটি সিস্টেমে চারটি শর্তে ডেডলক ঘটতে পারে। শর্তগুলো হলোঃ

i) Mutual Exclusion : ভিন্ন ভিন্ন সময়ে একটি রিসোর্সকে এক বা একাধিক প্রসেস ব্যবহার করতে পারে। কিন্তু যখন একাধিক প্রসেস একই সময়ে একই রিসোর্স ব্যবহার করে তখন ডেডলক অবস্থার সৃষ্টি হয়।

ii) Hold and Wait : একটি প্রসেস বিভিন্ন সময় একই সাথে একাধিক রিসোর্সকে ব্যবহার করার জন্য ধরে বা Hold রাখে। ফলে রিসোর্সটিকে আর অন্য কোনো প্রসেস ব্যবহার করতে পারে না। ফলে ডেডলক অবস্থা সৃষ্টি হয়।

iii) No-Preemption : কোনো প্রসেস কর্তৃক দখলকৃত কোনো রিসোর্স শুধুমাত্র প্রসেসটির কাজ সম্পন্ন হওয়ার পর মুক্ত করতে পারবে। অন্যথায় ডেডলক ঘটবে।

iv) Circular Wait : এক সেট ($P_0, P_1, P_2, \dots, P_n$) অপেক্ষমান প্রসেস এমনভাবে থাকবে যে P_0 অপেক্ষা করতে P_1 কর্তৃক দখলকৃত রিসোর্স এর জন্য, P_1 কর্তৃক দখলকৃত রিসোর্স এর জন্য P_2 অপেক্ষা করবে, P_3 কর্তৃক দখলকৃত রিসোর্স এর জন্য P_3 অপেক্ষা করবে, P_n কর্তৃক দখলকৃত রিসোর্সের জন্য P_{n-1} অপেক্ষা করবে এবং P_n অপেক্ষা করবে P_0 কর্তৃক দখলকৃত রিসোর্স এর জন্য। এভাবেই ডেডলক তৈরি হবে।

*** ২। **Deadlock** থেকে মুক্ত হওয়ার উপায়সমূহ কী কী?

NTRCA-24, বাকশি বো- ২০০৮, ১১, ১২, ১৪'পরি, ১৫, ১৮, ২০, ২৩, ২৪

উত্তরঃ কোনো System এ ডেডলক তৈরি হলে তা মুক্ত করা জরুরি। এক্ষেত্রে Deadlock Recovery করার জন্য দুটি পদ্ধতি রয়েছে। যথাঃ

১) **Process Termination :**

- i) যে সকল Process এর জন্য Deadlock ঘটতেছে সে সকল Process কে Abort করে দেয়া।
- ii) একই সময় একটি Process কে Abort করা যতক্ষণ না পর্যন্ত Deadlock Cycle শেষ হয়।

২) **Resource Preemption Method :** এ পদ্ধতিতে Deadlock Cycle এর এক বা একাধিক Resource এর Preemption ঘটিয়ে Deadlock Recovery করা যায়।

i) **Selection a victim :** যে Process টা সর্বোচ্চ Resource ধরে রেখেছে তাকে Select করে রাখা।

ii) **Roll back :** যে Process টি Select করে রেখেছি সেই Process টি Roll Back করতে হবে। অর্থাৎ সেই Process টি Safe State থেকে আবার Roll Back করতে পারি। অবশেষে Process টাকে Abort করে পুনরায় Restart করব।

iv) **Starvation :** যে Process টা Select করছি Victim হিসেবে সেই Process টাকে নির্দিষ্ট সংখ্যক সময় পর্যন্ত বিবেচনা করব।

*** ১। **Deadlock Prevention** এর বিভিন্ন স্তরগুলোর বর্ণনা কর।

বাকশিবো- ২০০৬, ১২, ১২'পরি

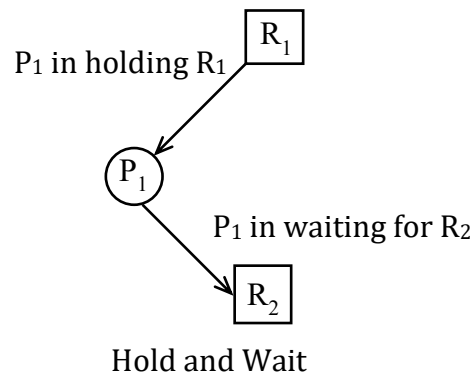
উত্তরঃ ডেডলক : একটি Resource কে যখন একাধিক প্রোগ্রাম বা Process ব্যবহার করতে চায় তখন যে অবস্থার সৃষ্টি হয়, অপারেটিং সিস্টেমের ভাষায় তাকে ডেডলক বলে।

ডেডলকের কিছু বৈশিষ্ট্য রয়েছে। যেমন :

- i) Mutual Exclusion
- ii) Hold and Wait
- iii) No Preemption এবং
- iv) Circular Wait

আমরা উপরের চারটি বৈশিষ্ট্যের যেকোনো একটিকে বাদ দিতে পারলে ডেডলক প্রতিরোধ করা সম্ভব।

Eliminate Mutual Exclusion: Mutual Exclusion সমস্যা দূর করা সম্ভব না, কারণ কিছু Resource যেমন টেপ ড্রাইভ এবং প্রিন্টার এগুলো একই সঙ্গে সহজভাবে শেয়ার করা যায় না। আবার Shareable Resource এর ক্ষেত্রে Mutual Exclusion এর দরকার পড়ে না। যেমন Read Only File অর্থাৎ একই সঙ্গে কতকগুলো Process কর্তৃক Read Only File গুলো ব্যবহার করতে পারে। Shareable Resource ব্যবহারের জন্য কোনো Process এর অপেক্ষা করতে হয় না।



Eliminate Hold and Wait: কোনো Process Execution শুরু করার আগে তার সকল প্রয়োজনীয় রিসোর্স বরাদ্দ করে যাতে Hold and Wait শর্তটি Eliminate হয়। এতে ডিভাইসের ব্যবহারকে কমিয়ে দেয়। উদাহরণস্বরূপ যদি একটি Process এর কিছু সময়ের পর একটি প্রিন্টারের প্রয়োজন হয় তাহলে আমরা Execution শুরু করার আগে একটি Printer কে বরাদ্দ করি। যাতে Printer টি Blocked থাকে Execution শেষ না হওয়া পর্যন্ত।

Eliminate No Preemption: এই শর্তটি ঘটবে না যদি একটি Process কিছু সংখ্যক রিসোর্সকে ধরে রাখে এবং অন্য আর একটি রিসোর্সের জন্য অনুরোধ করে যা তাৎক্ষণিকভাবে বরাদ্দ করা যায় না অর্থাৎ Process কে অপেক্ষা করা উচিত।

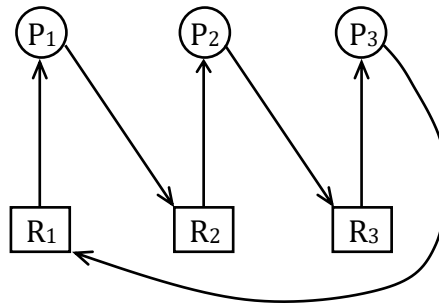
উদাহরণস্বরূপ :

i) Resource R_1 , R_2 এবং R_3 তে যদি একটি Process থাকে এবং নতুন Resource R_4 এর জন্য অপেক্ষা করে তবে এই অবস্থায় Resource সমূহকে (R_1 , R_2 , R_3 ও R_4) নতুন করে অনুরোধ করবে।

ii) যদি একটি Process P_1 কোনো Resource এর জন্য অপেক্ষা করে কিন্তু সেই Resource টিকে অন্য একটি Process P_2 ধরে রাখে এবং অন্য কোনো রিসোর্স এর জন্য অপেক্ষা করে। তারপর P_1 Process, P_2 Process এর ধরে রাখা Resource বরাদ্দ করে। এই পদ্ধতি সেই Resource এর জন্য সম্ভব যার অবস্থান সহজেই পুনরায় সংরক্ষণ করা যায়। যেমন : মেমরি, রেজিস্টার ইত্যাদি।

Eliminate Circular Wait: দুই বা ততোধিক Process বৃত্তের ক্রমানুসারে Resource এর জন্য অপেক্ষা করে, যা চিত্রের মাধ্যমে ভালোভাবে বুঝা যায় :

Circular Wait দূর করতে আমরা প্রত্যেক Resource কে অগ্রাধিকার দিয়ে থাকি। একটি Process শুধুমাত্র Resource এর অগ্রাধিকার ভিত্তিতে অনুরোধ করতে পারে।



উদাহরণ : উপরের চিত্রে আমরা দেখতে পাই P_3 Processor, Resource R_1 এর জন্য Request করেছে। যার একটি সংখ্যা রিসোর্স R_3 থেকে কম আছে যা ইতিমধ্যে P_3 Process করার জন্য বরাদ্দ করা হয়েছে। তাই এই Request টি অবৈধ, কারণ P_1 Process করার জন্য R_1 ইতিমধ্যেই বরাদ্দ করা হয়েছে।

*** ২। **Deadlock** দূরীকরণের **Algorithm** বর্ণনা কর।

36th BCS, বাকাশিবো- ২০০৪, ০৬, ০৮, ০৯, ১০, ১১, ১২, ১৪'পরী, ১৫, ১৬, ১৭, ১৮, ১৯, ২০, ২৪, ২৪'পরী

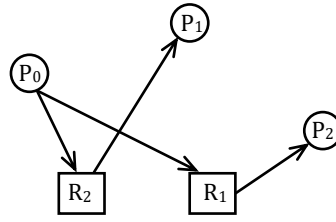
উত্তরঃ কোনো জটিল সিস্টেমে একাধিক Process এবং Resource সমূহ শেয়ার্ড অবস্থায় থাকে তাহলে Deadlock এর সম্ভাবনা দেখা দেয়, যখন Process গুলি অপেক্ষা করে একে অপরের Resource মুক্ত হওয়ার জন্য। ফলে ডেডলক কম্পিউটার সিস্টেমে গুরুতর সমস্যা সৃষ্টি করতে পারে, যেমন কর্মদক্ষতা হ্রাস করে এবং সিস্টেম ক্র্যাশ করে। এই ধরনের সমস্যা দূর করার জন্য ডেডলক এড়ানোর কৌশল প্রয়োগ করা হয়। এটি Process গুলো দ্বারা Resource এর জন্য Request করা এবং এই Request গ্রহণ করা হলে কোনো ডেডলক তৈরি হবে কিনা তা নির্ধারণ করার জন্য Resource সমূহ মূল্যায়ন করা। Deadlock দূর করা অপারেটিং সিস্টেম ডিজাইনের একটি গুরুত্বপূর্ণ দিক এবং কম্পিউটার সিস্টেমের নির্ভরযোগ্যতা এবং স্থিরতা বজায় রাখতে অপরিহার্য ভূমিকা পালন করে।

Safe State এবং Unsafe State: একটি Safe State এমন একটি সিস্টেম অবস্থাকে বোঝায় যেখানে প্রতিটি Process এর জন্য Resource allocate deadlock avoidance নিশ্চিত করে। এতে সকল Process এর execution সফলভাবে করা হয় এবং একটি Deadlock হওয়ার সম্ভাবনা কমে যায়। সিস্টেম একটি Safe State অর্জন করে যখন একটি উপযুক্ত ক্রমানুসারে সকল Process সফলভাবে Resource Allocate করতে সক্ষম হয়। অপরদিকে Unsafe State (এমন একটি সিস্টেম অবস্থাকে বুঝায়) যেখানে একটি Deadlock ঘটতে পারে। সকল

Process সফলভাবে Resource Allocate করতে পারে না এবং Deadlock হওয়ার সম্ভাবনা বেশি থাকে।

Deadlock দূর করার জন্য একটি জনপ্রিয় টেকনিক হিসেবে Resource Allocation Graph ব্যবহার করা হয়। এটি একটি Directed Graph যা সিস্টেমের মধ্যে Process গুলোকে Represent করে এবং তাদের মধ্যে সম্পর্ক থাকে। Resource allocation graph এ একটি Process Node এর দুই ধরনের Edge থাকে একটি হলো Request Edge অন্যটি হলো Assignment Edge. একটি Request Edge এ একটি Resource এর জন্য একটি Process দ্বারা একটি Request কে Represent করে। আবার যখন একটি Assignment দ্বারা Represent করে। সিস্টেমটি Safe State এ আছে কিনা তা নির্ধারণ করতে এবং Cycle Test করার জন্য Resource Allocation Graph বিশ্লেষণ করা হয়। যদি Graph এ একটি Cycle থাকে তাহলে সিস্টেম Unsafe State এ আছে এবং একটি Resource অনুরোধ গ্রহণ করার ফলে Deadlock হতে পারে। অপরদিকে Graph এ কোনো Cycle না থাকে তাহলে সিস্টেমটি Safe State অবস্থায় আছে এবং Deadlock ছাড়াই Resource Allocate করা যেতে পারে।

Resource allocation graph টেকনিকটি বাস্তবায়নের জন্য সিস্টেমের Process এবং Resource সমূহ পরিষ্কারভাবে উপস্থাপন করা হয়। এটি Deadlock শনাক্ত করার একটি কার্যকরী উপায় যদি Deadlock ঘটে থাকে। যাইহোক Resource Allocation Graph



Resource Allocation Graph

(RAG) টেকনিক এর প্রধান সীমাবদ্ধতা হলো বিশ্লেষণের শুরুতে সিস্টেমের সকল Resource সমূহ allocate করা হয়। অতএব এই সীমাবদ্ধতা কাটিয়ে উঠতে অন্যান্য কৌশল যেমন Bankers Algorithm ব্যবহার করা হয়।